Kyle Rauba - Novel Computer Vision Network Assignment - 4/7/2026

This novel computer vision network used a dataset of real images that was contained within a dataset that also had ai images. I decided to look at the real images because I felt it would be more consistent and actually useful for future analysis, but it would be cool to do this in the future with the ai dataset and see how the results compare with the same model architecture.

In [18]:
```
import pandas
import numpy
from torchvision import datasets, transforms
from torch.utils.data import random_split
from torch.utils.data import DataLoader

train_path = r'C:\Users\kyler\Data Science\Machine Learning\Computer Vision\real_da

train_transform = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ToTensor(),
])

# The data is transformed here, resized into 32 x 32 pixels, flipped and rotated ra
# It's then converted to a tensor so the network can actually accept it

full_data = datasets.ImageFolder(train_path, transform = transform)

train_size = int(0.7*len(full_data))
test_size = len(full_data) - (train_size)

test_data, train_data = random_split(full_data, [train_size, test_size])

# Test and training datasets are created from the real_images folder

train_loader = DataLoader(train_data, batch_size = 32, shuffle = True)
test_loader = DataLoader(test_data, batch_size = 32, shuffle = False)

full_data.classes

# This section loads the train and test loaders and establishes the model architect
# Referred to classes here to get an idea of the different classes
```

Out[18]:
```
['animals', 'city', 'food', 'nature', 'people']
```

In [19]:
```
import torch
import torch.nn as nn

class SeparableConv2d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size=3, stride=1, padding=
        super(SeparableConv2d, self).__init__()
        self.depthwise = nn.Conv2d(in_channels, in_channels, kernel_size=kernel_siz
```

```
                                    stride=stride, padding=padding, groups=in_channe
        self.pointwise = nn.Conv2d(in_channels, out_channels, kernel_size=1)

    def forward(self, x):
        x = self.depthwise(x)
        x = self.pointwise(x)
        return x

# Here is the first step of the actual architecture. The separable convolutions are
# The function is defined, and then the depthwise and pointwise convolutions are de
# The forward function tells the model to feed the data X through the model first t
```

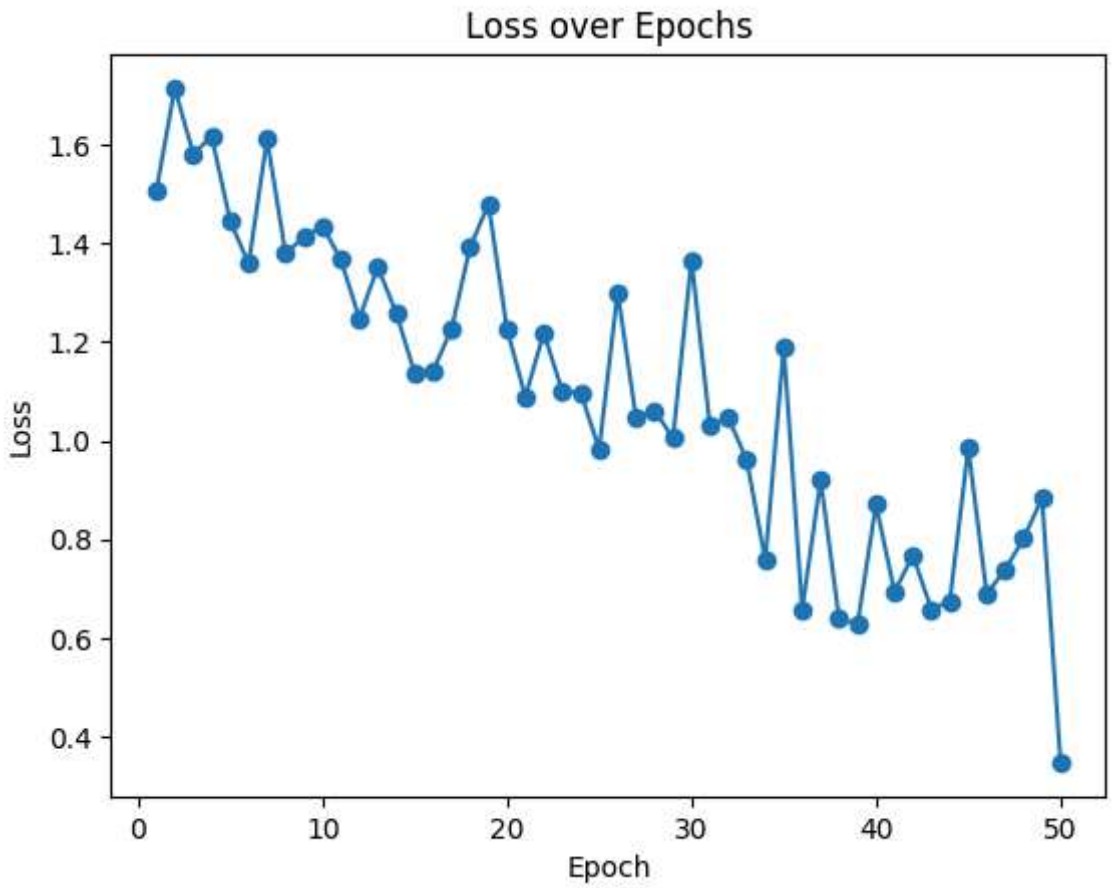
In [20]:

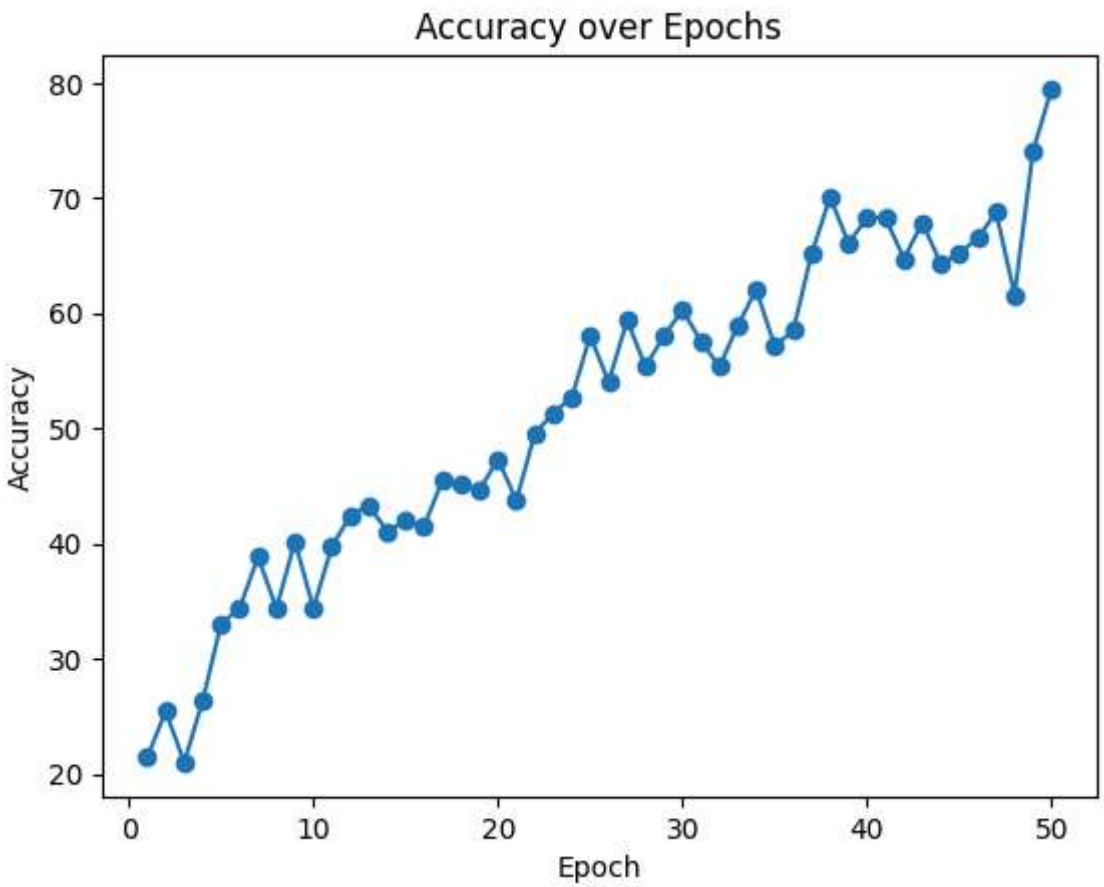
```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = SeparableConv2d(3, 16, 3)
        self.pool = nn.MaxPool2d(2)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(16 * 16 * 16, 512)
        self.relu1 = nn.ReLU()
        self.dropout = nn.Dropout(0.45)
        self.fc2 = nn.Linear(512, 256)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(256, 5)

# Here is the meat of the model architecture.
# Model is defined, and the model is initiated and referred to as self
# Super() is used to pull in functions from the parent function
# SeparableConv2d is called and placed at the start of the network, with 3 input ch
# A pooling layer compresses the size of the dimensions of the image, making it les
# A flatten layer is used to lay the pixels out side by side, making it 1D
# The first layer receives an input of 16 images (equal to the 16 output channels i
# 512 output channels are passed to the activation layer
# A dropout layer randomly turns off 45% of the neurons before the next layer
# The next layer receives 512 inputs and produces 256 outputs
# These 256 outputs are put through activation again to ensure non-linearity
# The last layer receives 256 inputs and produces 5 class outputs for the 5 classes
# Softmax specification isn't needed because it's built in to CategoricalCrossEntro

    def forward(self, x):
        x = self.conv1(x)
        x = self.pool(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.dropout(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
```

```
        return x

# Each layer is called 1 by 1 in the order listed above.
# The model is told to pass the data (X) through each layer as laid out above


model = Model()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.005)

# It took a bit of messing around to find a good learning rate for this model. 0.00

loss_values = []
accuracy_values = []

epochs = 50
for epoch in range(epochs):
    correct = 0
    total = 0
    for images, labels in train_loader:
        predictions = model(images)
        loss = criterion(predictions, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        _, predicted = torch.max(predictions, 1)
        correct += (predicted == labels).sum().item()
        total += labels.size(0)

    accuracy = correct/total * 100
    loss_values.append(loss.item())
    accuracy_values.append(accuracy)
    print(f"Epoch {epoch+1} - Loss: {loss.item():.4f} - Accuracy: {accuracy:.2f}%")

# This section creates 2 lists, loss_values and accuracy_values, for later plotting
# A for loop is created. For each epoch in the epochs range, start at a count of 0
# Our loss is calculated using our crossentropy established above, comparing the va
# We use zero_grad to zero out our calculations after every epoch, to avoid previou
# We then call loss backward and optimizer.step to backpropagate, our loss function
# Below that, we create our calculations for plotting. We pull our predictions out
# We add those values to our correct and total values, and calculate accuracy below

plt.plot(range(1, epochs + 1), loss_values, marker='o', label='Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Loss over Epochs')
plt.show()

plt.plot(range(1, epochs + 1), accuracy_values, marker='o', label='Training Accurac
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Accuracy over Epochs')
plt.show()

correct = 0
```

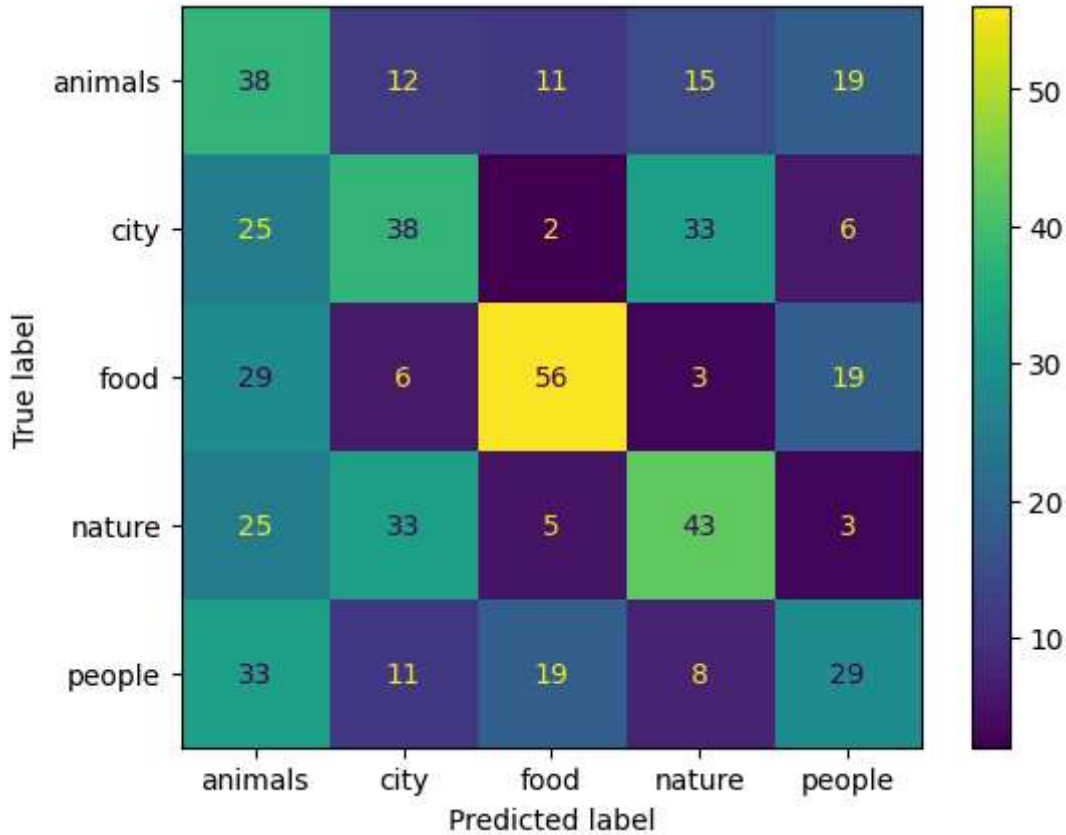
```
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        predictions = model(images)
        _, predicted = torch.max(predictions, 1)
        correct += (predicted == labels).sum().item()
        total += labels.size(0)
    sample_images, sample_labels = next(iter(train_loader))
    sample_preds = model(sample_images)
    print(sample_preds[:3])
    print(sample_labels[:3])

# Similar process as above, but this allows us to view our total test accuracy, as

print(f"Test Accuracy: {correct / total * 100:.2f}%")

all_predictions = []
all_labels = []

with torch.no_grad():
    for images, labels in test_loader:
        predictions = model(images)
        _, predicted = torch.max(predictions, 1)
        all_predictions.extend(predicted.numpy())
        all_labels.extend(labels.numpy())

matrix = confusion_matrix(all_labels, all_predictions)
display = ConfusionMatrixDisplay(matrix, display_labels = full_data.classes)
display.plot()
plt.show()

# Here is where we create our confusion matrix, by creating all_predictions and all
# These lists are appended in a similar way as above, with a for loop and extending
# confusion_matrix and ConfusionMatrixDisplay were imported from sklearn, and visua
```

```
Epoch 1 - Loss: 1.5062 - Accuracy: 21.43%
Epoch 2 - Loss: 1.7146 - Accuracy: 25.45%
Epoch 3 - Loss: 1.5816 - Accuracy: 20.98%
Epoch 4 - Loss: 1.6166 - Accuracy: 26.34%
Epoch 5 - Loss: 1.4458 - Accuracy: 33.04%
Epoch 6 - Loss: 1.3585 - Accuracy: 34.38%
Epoch 7 - Loss: 1.6116 - Accuracy: 38.84%
Epoch 8 - Loss: 1.3800 - Accuracy: 34.38%
Epoch 9 - Loss: 1.4122 - Accuracy: 40.18%
Epoch 10 - Loss: 1.4353 - Accuracy: 34.38%
Epoch 11 - Loss: 1.3668 - Accuracy: 39.73%
Epoch 12 - Loss: 1.2480 - Accuracy: 42.41%
Epoch 13 - Loss: 1.3522 - Accuracy: 43.30%
Epoch 14 - Loss: 1.2589 - Accuracy: 41.07%
Epoch 15 - Loss: 1.1367 - Accuracy: 41.96%
Epoch 16 - Loss: 1.1419 - Accuracy: 41.52%
Epoch 17 - Loss: 1.2273 - Accuracy: 45.54%
Epoch 18 - Loss: 1.3921 - Accuracy: 45.09%
Epoch 19 - Loss: 1.4795 - Accuracy: 44.64%
Epoch 20 - Loss: 1.2264 - Accuracy: 47.32%
Epoch 21 - Loss: 1.0893 - Accuracy: 43.75%
Epoch 22 - Loss: 1.2189 - Accuracy: 49.55%
Epoch 23 - Loss: 1.0999 - Accuracy: 51.34%
Epoch 24 - Loss: 1.0967 - Accuracy: 52.68%
Epoch 25 - Loss: 0.9828 - Accuracy: 58.04%
Epoch 26 - Loss: 1.3010 - Accuracy: 54.02%
Epoch 27 - Loss: 1.0453 - Accuracy: 59.38%
Epoch 28 - Loss: 1.0614 - Accuracy: 55.36%
Epoch 29 - Loss: 1.0052 - Accuracy: 58.04%
Epoch 30 - Loss: 1.3662 - Accuracy: 60.27%
Epoch 31 - Loss: 1.0305 - Accuracy: 57.59%
Epoch 32 - Loss: 1.0466 - Accuracy: 55.36%
Epoch 33 - Loss: 0.9603 - Accuracy: 58.93%
Epoch 34 - Loss: 0.7576 - Accuracy: 62.05%
Epoch 35 - Loss: 1.1893 - Accuracy: 57.14%
Epoch 36 - Loss: 0.6584 - Accuracy: 58.48%
Epoch 37 - Loss: 0.9201 - Accuracy: 65.18%
Epoch 38 - Loss: 0.6401 - Accuracy: 70.09%
Epoch 39 - Loss: 0.6283 - Accuracy: 66.07%
Epoch 40 - Loss: 0.8707 - Accuracy: 68.30%
Epoch 41 - Loss: 0.6939 - Accuracy: 68.30%
Epoch 42 - Loss: 0.7674 - Accuracy: 64.73%
Epoch 43 - Loss: 0.6586 - Accuracy: 67.86%
Epoch 44 - Loss: 0.6746 - Accuracy: 64.29%
Epoch 45 - Loss: 0.9847 - Accuracy: 65.18%
Epoch 46 - Loss: 0.6897 - Accuracy: 66.52%
Epoch 47 - Loss: 0.7387 - Accuracy: 68.75%
Epoch 48 - Loss: 0.8018 - Accuracy: 61.61%
Epoch 49 - Loss: 0.8834 - Accuracy: 74.11%
Epoch 50 - Loss: 0.3479 - Accuracy: 79.46%
```


Loss over Epochs
Loss
Epoch
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
10
20
30
40
50


Accuracy over Epochs
Accuracy
Epoch
80
70
60
50
40
30
20
0
10
20
30
40
50

```
tensor([[-0.4717,  0.7056, -1.4560,  0.5234, -0.3470],
        [ 3.2595, -3.2083, -2.0828, -1.4346,  0.6078],
        [-0.1311, -5.8949,  7.3785, -2.5057,  2.9997]])
tensor([1, 0, 2])
Test Accuracy: 42.99%
```



In [ ]: